

RFM Customer Segmentation Using Unsupervised Machine Learning

Problem Statement & Solution Approach

1. Problem Statement

In e-commerce, businesses accumulate millions of transactional records daily, yet most marketing strategies treat the entire customer base as a single homogeneous group. This one-size-fits-all approach leads to wasted marketing spend, poor customer retention, and missed revenue opportunities. Without a data-driven understanding of customer behaviour patterns, businesses cannot identify which customers are their most valuable, which are at risk of churning, and which have already become dormant.

Formal Problem Statement: Given a transactional dataset of 100,000+ e-commerce orders from the Olist Brazilian marketplace, develop an end-to-end unsupervised machine learning pipeline that (1) engineers Recency, Frequency, and Monetary (RFM) behavioural features from raw transaction logs, (2) reduces dimensionality via Principal Component Analysis to produce an orthogonal, visualisable feature space, (3) discovers natural customer segments through K-Means clustering with rigorous hyperparameter optimisation, and (4) profiles each segment with interpretable business labels and SHAP-based explainability, enabling targeted marketing strategies.

2. What Problem Are We Trying to Solve?

The project addresses three interconnected challenges that e-commerce businesses face when trying to understand and act on customer behaviour:

2.1 Customer Heterogeneity Is Invisible in Raw Data

Raw transactional tables contain order timestamps, payment amounts, and product IDs, but they do not reveal behavioural patterns. A customer who placed one large order six months ago looks identical in a flat table to a customer who places small orders every week. Without engineered behavioural features, there is no axis along which to compare customers meaningfully.

- **Recency** — How many days since the customer's last purchase? A proxy for engagement. Customers who bought recently are more likely to buy again.
- **Frequency** — How many distinct orders has the customer placed? Measures loyalty and repeat-purchase behaviour.
- **Monetary** — What is the customer's total lifetime spend? Captures economic value to the business.

These three metrics together capture the full spectrum of customer value: when they buy, how often they buy, and how much they spend.

2.2 High-Dimensional, Skewed Features Resist Direct Clustering

Raw RFM distributions are heavily right-skewed (e.g., monetary skewness = 3.17 in this dataset) and contain extreme outliers (max monetary = €13,664 vs median = €108). Clustering algorithms like K-Means rely on Euclidean distance, which is distorted by skewed scales — a single high-spend outlier can pull an entire cluster centroid. Additionally, correlated features (frequency and monetary, $r = 0.16$) introduce redundant dimensions that dilute cluster separation.

- **Skewness** must be reduced through power transforms (Box-Cox / Yeo-Johnson) before standardisation.
- **Outliers** must be capped (not dropped) to preserve sample size while bounding extreme values.
- **Correlation** between features must be resolved through PCA to produce orthogonal axes that maximise variance in fewer dimensions.

2.3 The Number of Segments Is Unknown

Unlike supervised learning where labels are given, customer segmentation is an unsupervised problem — we do not know in advance how many natural groups exist. Choosing too few clusters merges distinct behaviours; too many creates unactionable micro-segments. The optimal K must be determined empirically using multiple validation metrics (Silhouette Score, Calinski-Harabasz Index, Davies-Bouldin Index) and compared against density-based alternatives (DBSCAN) to ensure the chosen algorithm fits the data's geometry.

2.4 Clusters Must Be Interpretable, Not Just Statistical

A cluster label like "Cluster 0" is meaningless to a marketing team. Each discovered segment needs a human-readable business label (Champions, Potential Loyalists, At-Risk Spenders, Hibernating) and quantitative profiles showing what drives membership. SHAP explainability via a proxy Random Forest classifier reveals which RFM features most influence each segment's identity, turning statistical clusters into actionable business intelligence.

3. How Are We Trying to Solve It?

The solution is a four-phase production Python pipeline, where each phase addresses one of the challenges above. The phases execute sequentially, with each phase's output feeding directly into the next:

Phase 1: RFM Feature Engineering

Goal: Transform 100K raw transaction records into 93,357 customer-level behavioural profiles.

- **Data loading & joining** — Three CSV files (orders, customers, payments) are joined on `order_id` and `customer_id`. Only delivered orders are retained (96,478 of 99,441).
- **RFM aggregation** — For each unique customer: Recency = days since last purchase (relative to 18 Oct 2018), Frequency = count of distinct orders, Monetary = sum of all payment values.

- **Outlier capping** — Values beyond the 1st and 99th percentiles are clipped (not dropped), preserving all 93,357 customers while bounding extremes.
- **Power transforms** — Box-Cox is applied to recency ($\lambda=0.45$, skew $0.42 \rightarrow 0.09$) and monetary ($\lambda=-0.19$, skew $3.17 \rightarrow 0.02$). Frequency is skipped (only 2 unique values after capping — 97% of customers are one-time buyers).
- **Z-score standardisation** — StandardScaler centres all features to mean=0, std=1, ensuring equal contribution to distance-based algorithms.

Output: rfm_raw.csv (original-scale profiles for later interpretation) and rfm_scaled.csv (transformed + standardised features for PCA/clustering).

Phase 2: PCA Dimensionality Reduction

Goal: Decorrelate features and project into a 2D space for clustering and visualisation.

- **Correlation analysis** — Post-transform correlation matrix confirms features are weakly correlated (max $|r| = 0.16$), meaning variance is spread across all 3 components.
- **Full PCA decomposition** — PCA is fit once on all 3 components, then sliced to 2. This avoids redundant refitting while providing the complete scree plot for documentation.
- **Variance retention** — 2 components capture 72% of total variance. While below the conventional 90% threshold, this is because the features are genuinely weakly correlated after transforms — not a sign of information loss. The 72% is sufficient for cluster separation (silhouette = 0.41).
- **Component interpretation** — PC1 = spending behaviour (frequency + monetary loadings ≈ 0.70 each), PC2 = recency (loading = 0.99). This gives the scatter plot meaningful axes.

Output: pca_transformed.csv (93,357 customers $\times$ 2 principal components), pca_loadings.csv, pca_variance.csv.

Phase 3: K-Means Clustering with DBSCAN Benchmark

Goal: Discover the optimal number of customer segments and assign every customer to a cluster.

- **Multi-metric elbow analysis** — K-Means is run for K=2 through K=10. Four metrics are computed at each K: Inertia (WCSS), Silhouette Score, Calinski-Harabasz Index, Davies-Bouldin Index.
- **Optimal K selection** — K=3 is selected as the silhouette-optimal value (score = 0.41). Silhouette is used as the primary metric because it directly measures cluster cohesion vs separation without requiring ground-truth labels.
- **Final K-Means fit** — K-Means with K=3, n_init=20 (20 random initialisations to avoid local minima), deterministic seed. Cluster sizes: 42,865 / 2,801 / 47,691 customers.
- **DBSCAN comparison** — A density-based benchmark is run with multi-eps grid search across the 75th–95th percentiles of k-NN distances. Best DBSCAN result: 5 clusters, silhouette = 0.20. K-Means clearly outperforms (0.41 vs 0.20), confirming the data has roughly spherical cluster geometry that suits centroid-based methods.

Output: clustered_customers.csv (each customer's cluster label), clustering_metrics.csv, cluster_centroids.csv, algorithm_comparison.csv.

Phase 4: Segment Profiling & SHAP Explainability

Goal: Translate statistical clusters into actionable business segments with interpretable labels and feature-importance explanations.

- **Profile table** — Cluster labels are merged back to original-scale RFM values (not transformed/scaled) so that profiles are in human-readable units (days, order count, currency).
- **Segment labelling** — Each cluster's RFM means are normalised to [0, 1] across clusters. A recency score (inverted: lower recency = higher score) and a value score (average of normalised frequency + monetary) determine the label:
 - **Champions** — Recent + high value ($R \geq 0.5$, $V \geq 0.5$)
 - **Potential Loyalists** — Recent + low value ($R \geq 0.5$, $V < 0.5$)
 - **At-Risk Spenders** — Not recent + high value ($R < 0.5$, $V \geq 0.5$)
 - **Hibernating** — Not recent + low value ($R < 0.5$, $V < 0.5$)
- **SHAP explainability** — A Random Forest proxy classifier is trained on original RFM features to predict cluster membership (5-fold CV accuracy: 99.99%). SHAP TreeExplainer then quantifies each feature's contribution to each cluster's identity. Key finding: recency is the dominant driver (SHAP = 0.478) for separating Potential Loyalists from Hibernating, while frequency distinguishes Champions (SHAP = 0.057).

Output: segment_profiles.csv, shap_importance.csv, plus 8 diagnostic and presentation plots.

4. Discovered Segments

The pipeline identifies three distinct customer segments:

Segment	Count	%	Recency (days)	Frequency (orders)	Monetary (€)
Potential Loyalists	42,865	45.9%	155	1.0	154
Champions	2,801	3.0%	269	2.0	295
Hibernating	47,691	51.1%	405	1.0	156

Key insight: 97% of Olist customers are one-time buyers (frequency = 1), making recency the primary differentiator between segments. The 3% who return (Champions) spend nearly 2× more per visit and represent the highest lifetime value. This directly informs strategy: retention campaigns for Potential Loyalists, win-back campaigns for Hibernating, and loyalty rewards for Champions.

5. Technology Stack

Layer	Tools
Data Processing	pandas, NumPy
Statistical Transforms	SciPy (Box-Cox, Yeo-Johnson)

Layer	Tools
Machine Learning	scikit-learn (PCA, K-Means, DBSCAN, StandardScaler, RandomForest)
Explainability	SHAP (TreeExplainer)
Visualisation	Matplotlib, Seaborn
Runtime	Python 3.9+, modular CLI pipeline